Лабораторная работа № 17

Регистрация и авторизация пользователей. Хранение токена

О чём лабораторная?

До этого момента приложение уже умеет получать объявления с сервера, но ещё не использует полноценную авторизацию. Добавим регистрацию и вход через API, научимся получать токены, сохранять их на устройстве и автоматически прикреплять токен к запросам, которым нужна авторизация.

В результате пользователь сможет зарегистрироваться, войти в аккаунт, перезапустить приложение и продолжить работу без повторного ввода пароля.

Часть 1. Цель работы

Научиться:

- изучать API регистрации и авторизации
- создавать модели запросов и ответов
- отправлять POST-запросы с помощью Retrofit
- обрабатывать успешные ответы и ошибки сервера
- получать `access_token` и `refresh_token`
- сохранять и удалять токены
- добавлять заголовок `Authorization: Bearer ...` к защищённым запросам
- проверять авторизацию через профиль текущего пользователя
- восстанавливать сессию при следующем запуске приложения

Основные понятия

Регистрация - создание нового аккаунта

Авторизация - вход пользователя с использованием учётных данных

Access token - токен, который приложение передаёт серверу для доступа к защищённым операциям

Refresh token - токен, предназначенный для получения новой пары токенов

Bearer-токен - токен, передаваемый в HTTP-заголовке:

```
Authorization: Bearer ACCESS_TOKEN
```

Токен - не пароль. После успешного входа приложение должно использовать токен, а не сохранять пароль для повторной отправки.

Часть 2. Изучаем API

Шаг 1. Откройте документацию

Перейдите по адресу:

`http://95.31.156.238:8000/docs`

Найдите операции:

- `POST /auth/register` - регистрация
- `POST /auth/login` - вход
- `POST /auth/refresh` - обновление токенов
- `GET /user/me` - получение данных текущего пользователя

Последняя операция требует авторизации через Bearer-токен.

Шаг 2. Изучите запрос регистрации

Сервер ожидает JSON следующего вида:

```
{
  "username": "student_01",
  "email": "student@example.com",
  "password": "StrongPass123"
}
```

Требования нашего API:

Поле	Ограничения
`username`	3–32 символа, латинские буквы, цифры и `_`
`email`	Корректный email, не более 255 символов
`password`	8–64 символа

Успешная регистрация возвращает статус `201 Created` и объект пользователя с полями `id`, `username`, `about_me`

Важно: ответ регистрации не содержит токены. После регистрации нужно отдельно выполнить вход через `/auth/login`

Шаг 3. Изучите запрос авторизации

Тело запроса:

```
{
  "email": "student@example.com",
  "password": "StrongPass123"
}
```

Успешный ответ с токеном содержит примерно такое:

```
{
  "access_token": "abcd123abcd123abcd123abcd123",
  "refresh_token": "efgh456efgh456efgh456efgh456",
  "token_type": "bearer"
}
```

Шаг 4. Изучите защищённый запрос

Операция `GET /user/me` не требует тела, но требует заголовка:

```
Authorization: Bearer abcd123abcd123abcd123abcd123
```

Если токен отсутствует или недействителен, сервер может отклонить запрос. Приложение должно корректно обработать такой ответ, а не завершаться с ошибкой.

Часть 3. Создаём модели запросов и ответов

Самостоятельно создайте классы для запросов `RegisterRequest`, `LoginRequest` и для ответов `TokenResponse`, `UserResponse`.

Имена полей должны обязательно совпадать совпадают с JSON, чтобы `Gson` сопоставил их автоматически.

Часть 4. Добавляем методы в `ApiService`

Откройте существующий `ApiService.java` и добавьте в него методы регистрации, входа и получения текущего пользователя.

```
public interface ApiService {

    @POST("auth/register")
    Call<UserResponse> register(
            @Body RegisterRequest request
    );

    @POST("auth/login")
    Call<TokenResponse> login(
            @Body LoginRequest request
    );

    @GET("user/me")
    Call<UserResponse> getMe();
}
```

Если в `ApiService` уже есть метод `getItems()`, **оставьте его**. Не удаляйте существующие методы загрузки объявлений.

Разберём аннотации

- `@POST("auth/register")` - отправляет POST-запрос на регистрацию
- `@Body` - передаёт объект в теле запроса, который Gson преобразует в JSON
- `Call<UserResponse>` - ожидаемый ответ регистрации
- `Call<TokenResponse>` - ожидаемый ответ авторизации
- `@GET("user/me")` - запрашивает данные текущего пользователя

На этом этапе методы описаны, но токен ещё не сохраняется и не прикрепляется к запросам.

Часть 5. Создаём хранилище токенов

Для проекта используем `SharedPreferences`, но важно понимать, что это упрощённый вариант хранения: обычный `SharedPreferences` **не шифрует данные автоматически**, потому он не особо безопасен для их хранения.

Создайте `TokenStorage.java`.

```
public class TokenStorage {

    private static final String PREFS_NAME = "auth_prefs";
    private static final String KEY_ACCESS_TOKEN = "access_token";
    private static final String KEY_REFRESH_TOKEN = "refresh_token";

    private final SharedPreferences prefs;

    public TokenStorage(Context context) {
        prefs = context.getApplicationContext()
                .getSharedPreferences(
                        PREFS_NAME,
                        Context.MODE_PRIVATE
                );
    }

    public void saveTokens(
        String accessToken,
        String refreshToken
    ) {
        prefs.edit()
                .putString(KEY_ACCESS_TOKEN, accessToken)
                .putString(KEY_REFRESH_TOKEN, refreshToken)
                .apply();
    }

    public String getAccessToken() {
        return prefs.getString(KEY_ACCESS_TOKEN, null);
    }

    public String getRefreshToken() {
        return prefs.getString(KEY_REFRESH_TOKEN, null);
    }
```

```
    public boolean hasAccessToken() {
        String token = getAccessToken();
        return token != null && !token.isEmpty();
    }

    public void clearTokens() {
        prefs.edit().clear().apply();
    }
}
```

Как работает класс?

- `saveTokens()` сохраняет оба токена
- `getAccessToken()` возвращает токен доступа
- `getRefreshToken()` возвращает токен обновления
- `hasAccessToken()` помогает проверить, есть ли сохранённый токен
- `clearTokens()` удаляет сохранённые данные авторизации

Часть 6. Добавляем токен к HTTP-запросам

Чтобы не добавлять заголовок *вручную* в каждом методе, используем **OkHttp Interceptor** или же "перехватчик".

Откройте и дополните `RetrofitClient.java`:

```
public class RetrofitClient {

    private static final String BASE_URL = "http://95.31.156.238:8000/";

    private static Retrofit retrofit;

    // наш метод теперь принимает контекс, так как
    // он нужен для хранилища токенов
    public static Retrofit getRetrofit(Context context) {
        if (retrofit == null) {
            HttpLoggingInterceptor logging = new HttpLoggingInterceptor();
            logging.setLevel(
                    HttpLoggingInterceptor.Level.BODY
            );

            // получаем хранилище токенов
            TokenStorage tokenStorage = new TokenStorage(context);

            // создаем перехватчик
            Interceptor authInterceptor = chain -> {
                Request original = chain.request();
                String token = tokenStorage.getAccessToken();

                Request.Builder builder = original.newBuilder();
```

```
                // если токен не пустой, то прикрепляем его к запрсоу
                if (token != null && !token.isEmpty()) {
                    builder.header(
                        "Authorization",
                        "Bearer " + token
                    );
                }

                return chain.proceed(builder.build());
            };

            OkHttpClient client = new OkHttpClient.Builder()
                    .addInterceptor(logging)
                    .addInterceptor(authInterceptor) // не забываем прикрепить
перехватчик
                    .build();

            retrofit = new Retrofit.Builder()
                    .baseUrl(BASE_URL)
                    .client(client)
                    .addConverterFactory(
                            GsonConverterFactory.create()
                    )
                    .build();
        }

        return retrofit;
    }

    // соответственно тут тоже создаем хранилище токенов
    public static ApiService getApiService(Context context) {
        return getRetrofit(context).create(ApiService.class);
    }
}
```

Обратите внимание, что раньше клиент мог создаваться через `getRetrofit()` без параметров. Так как ему теперь нужен `Context` - вызовы нужно обновить, например:

```
ApiService apiService = RetrofitClient.getApiService(getApplicationContext());
```

Часть 7. Реализуем регистрацию

Шаг 1. Подготовьте экран регистрации

Используйте существующий экран регистрации из проекта:

Поля:

- имя пользователя (`username`)

- email
- пароль
- повтор пароля
- кнопка "Зарегистрироваться"

Шаг 2. Проверьте данные на устройстве

Перед запросом проверьте:

- имя пользователя содержит от 3 до 32 символов
- имя состоит только из латинских букв, цифр и _
- email соответствует формату адреса
- пароль содержит от 8 до 64 символов
- повтор пароля совпадает с паролем

Для email можно использовать:

```
Patterns.EMAIL_ADDRESS.matcher(email).matches()
```

Для имени можно использовать регулярное выражение:

```
username.matches("[a-zA-Z0-9_]{3,32}")
```

Шаг 3. Отправьте запрос регистрации

```
ApiService apiService = RetrofitClient.getApiService(getApplicationContext());

RegisterRequest request = new RegisterRequest(
    username,
    email,
    password
);

apiService.register(request).enqueue(new Callback<UserResponse>() {

    @Override
    public void onResponse(
        Call<UserResponse> call,
        Response<UserResponse> response
    ) {
        if (response.isSuccessful() && response.body() != null) {

            // регистрация завершена успешно - можно делать логин

        } else {
            // запрос не успешен - можете показать сообщение об ошибке
        }
    }
```

```
    @Override
    public void onFailure(
        Call<UserResponse> call,
        Throwable t
    ) {
        // нет соединения или запрос не выполнен
    }
});
```

После успешной регистрации выполните вход с email и паролем, которые пользователь только что ввёл. Не считайте пользователя авторизованным, **пока не получены токены.**

Часть 8. Реализуем авторизацию и сохраняем токены

Шаг 1. Отправьте запрос входа

```
ApiService apiService = RetrofitClient.getApiService(getApplicationContext());

LoginRequest request = new LoginRequest(email, password);

apiService.login(request).enqueue(new Callback<TokenResponse>() {

    @Override
    public void onResponse(
        Call<TokenResponse> call,
        Response<TokenResponse> response
    ) {
        if (response.isSuccessful() && response.body() != null) {

            TokenResponse tokens = response.body();

            TokenStorage tokenStorage =
                    new TokenStorage(getApplicationContext());

            tokenStorage.saveTokens(
                    tokens.getAccessToken(),
                    tokens.getRefreshToken()
            );

            // токены сохранены
            // теперь можно открыть основной экран

        } else {
            // неверные данные или другая ошибка
        }
    }

    @Override
    public void onFailure(
        Call<TokenResponse> call,
        Throwable t
```

```
    ) {
        // ошибка сети
    }
});
```

Шаг 3. Перейдите на главный экран

После успешного входа и сохранения токенов:

1. Закройте экран входа
2. Откройте основной экран приложения
3. Не позволяйте кнопке "Назад" возвращать пользователя на форму входа, если это противоречит навигации проекта

Используйте существующую структуру экранов и навигацию, не создавайте дубликаты `MainActivity`.

Требования

- Созданы модели запросов регистрации и входа
- Создана модель ответа с токенами
- Регистрация работает через `POST /auth/register`
- После регистрации выполняется вход для получения токенов
- Авторизация работает через `POST /auth/login`
- Токены сохраняются и читаются через `TokenStorage`
- Интерсептор добавляет Bearer-токен к защищённым запросам
- Запрос `GET /user/me` выполняется успешно после входа
- При запуске приложение проверяет наличие сохранённого access-токена
- Если токен есть, приложение проверяет его через `GET /user/me` и выбирает экран в зависимости от результата

Дополнительно

1. Реализуйте обновление токенов через `POST /auth/refresh`
2. Добавьте индикатор загрузки на экраны входа и регистрации
3. Показывайте сообщения об ошибках рядом с соответствующими полями формы
4. После восстановления сессии загружайте профиль пользователя и отображайте его имя на экране профиля